

Configurar OpenCode para comportamiento consistente entre modelos

Archivos clave que debes modificar

OpenCode usa una estructura de configuración que aplica **reglas globales** sin importar el modelo:

1. **opencode.json (Configuración principal)**

Este archivo va en la **raíz de tu proyecto**:

JSON

```
{
  "provider": "anthropic",
  "model": "claude-sonnet-4-20250514",
  "providers": {
    "anthropic": {
      "apiKey": "sk-ant-..."
    },
    "openai": {
      "apiKey": "sk-..."
    },
    "openrouter": {
      "apiKey": "sk-or-..."
    }
  }
}
```

2. **Archivo de reglas (LO MÁS IMPORTANTE)**

Crea un archivo de **instrucciones/reglas** que OpenCode inyecta como system prompt a **cualquier modelo**:

Opción A: .opencode/rules.md

Markdown

Reglas Globales

Idioma

- SIEMPRE responde en español

- Usa comentarios en español en el código

Estilo de código

- Usa TypeScript estricto
- Siempre agrega tipos explícitos
- Nunca uses `any`
- Prefiere funciones puras
- Agrega manejo de errores siempre

Comportamiento

- No expliques lo que vas a hacer, HAZLO directamente
- Si modificas un archivo, muestra el archivo COMPLETO
- Sigue los patrones existentes del proyecto
- No cambies dependencias sin preguntar
- Siempre verifica que el código compile antes de darlo por terminado

Estructura del proyecto

- Frontend: React + Vite
- Backend: Node.js + Express
- Base de datos: PostgreSQL
- ORM: Prisma

Prohibiciones

- NUNCA borres tests existentes
- NUNCA cambies la configuración de ESLint
- NUNCA instales dependencias sin confirmación

Opción B: CONVENTIONS.md en la raíz

Markdown

Convenciones del Proyecto

Stack tecnológico

- Python 3.12
- FastAPI
- SQLAlchemy
- Pydantic v2

Reglas obligatorias

1. Toda función pública debe tener docstring

2. Usa type hints en TODOS los parámetros
3. Los endpoints deben tener validación con Pydantic
4. Maneja excepciones con HTTPException
5. Escribe tests para cada endpoint nuevo

Estructura de archivos

src/

- |—— api/ # Endpoints
- |—— models/ # SQLAlchemy models
- |—— schemas/ # Pydantic schemas
- |—— services/ # Lógica de negocio
- |—— utils/ # Utilidades

text

Ejemplo de código que DEBES seguir

```
```python
```

```
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel
```

```
class UserCreate(BaseModel):
 """Schema para crear usuario."""
 name: str
 email: str
```

```
@router.post("/users", response_model=UserResponse)
async def create_user(data: UserCreate) -> UserResponse:
 """Crea un nuevo usuario."""
 try:
 user = await user_service.create(data)
 return UserResponse.model_validate(user)
 except ValueError as e:
 raise HTTPException(status_code=400, detail=str(e))
```

text

---

## 3.  `.env` (Variables de entorno)

```
```bash
# Proveedores
ANTHROPIC_API_KEY=sk-ant-xxxxx
OPENAI_API_KEY=sk-xxxxx
OPENROUTER_API_KEY=sk-or-xxxxx

# Modelo por defecto
OPENCODE_MODEL=claude-sonnet-4-20250514
OPENCODE_PROVIDER=anthropic
```

4. .cursorrules / .windsurfrules (compatibilidad)

OpenCode también puede leer estos archivos que otros editores usan:

Markdown

Eres un programador senior trabajando en este proyecto.

REGLAS INQUEBRANTABLES:

1. Todo código debe estar en español (comentarios y variables)
2. Responde SIEMPRE en español
3. Antes de modificar algo, explica el impacto
4. Si no sabes algo, dilo honestamente
5. Sigue el patrón existente del código

Estructura final de tu proyecto

text

```
mi-proyecto/
├── .opencode/
│   ├── rules.md      ← TUS REGLAS GLOBALES
│   ├── .env          ← API keys
│   ├── opencode.json  ← Configuración del tool
│   ├── CONVENTIONS.md ← Convenciones del proyecto
│   └── src/
│       └── ...
```

 **Consejos para que CUALQUIER modelo obedezca igual**

Sé específico y estructurado en tus reglas:

Markdown

❌ MAL (vago, ambiguo)

"Escribe buen código y sé útil"

✅ BIEN (específico, con ejemplos)

"Toda función debe:

- 1. Tener máximo 20 líneas
- 2. Un solo propósito
- 3. Retornar tipos explícitos
- 4. Ejemplo:

```
```python
def calcular_total(items: list[Item]) -> Decimal:
 return sum(item.precio for item in items)
```
```

Usa formato de prioridades:

Markdown

Prioridad CRÍTICA (nunca ignorar)

- No borrar archivos sin confirmación
- No cambiar esquema de BD sin migración

Prioridad ALTA

- Seguir convenciones de nombres
- Agregar tests

Prioridad MEDIA

- Optimizar imports
- Documentar funciones complejas

Resumen

| Archivo | Para qué sirve | Impacto |
|--------------------|---------------------------------------|----------|
| .opencode/rules.md | Reglas inyectadas a TODOS los modelos | 🔴 Máximo |

| | | |
|----------------|-----------------------------------|------------------|
| opencode.json | Config de
proveedores y modelo | 🟡 Medio |
| .env | API keys | 🟡 Medio |
| CONVENTIONS.md | Contexto del proyecto | 🟢 Complementario |